

ENGR 3590 - Kinematics and Dynamics of Machinery

Project 3: 6-Bar Pick-and-Place Robot

Middle Tennessee State University

April 28 2026

Abstract

This report details the design, kinematic analysis, and MATLAB simulation of an industrial 6-bar pick-and-place robot featuring a prismatic joint. The objective was to design a mechanism capable of transferring objects from a high conveyor to a sloped drop-off zone. This document outlines the rationale behind utilizing Inverse Kinematics over Forward Kinematics, details the iterative design process that resolved various mathematical and physical singularities, and documents the MATLAB implementation used to generate the final simulation.

1 Introduction

The primary objective of Project 3 was to design a 6-bar linkage system equipped with a prismatic joint to function as an automated pick-and-place cell. The system must navigate from a precise pick location (a high conveyor grating) to a dynamic drop location (a sloped lower conveyor) without exceeding the physical reach limits of the mechanism.

2 Kinematic Analysis: Inverse vs. Forward Kinematics

2.1 The Necessity of Inverse Kinematics

A critical design decision in this project was structuring the trajectory generation entirely around **Inverse Kinematics (IK)** rather than Forward Kinematics (FK).

Forward Kinematics calculates the final position of the end-effector (X, Y) given known joint angles (θ_2, θ_5) . In a pick-and-place task, the target coordinates are strictly dictated by the environment (e.g., the top of the wooden table or the exact sloped line of a conveyor belt). If FK were used, reaching these exact (X, Y) target pixels would require an iterative, trial-and-error approach to guess the required crank angles. This approach is computationally expensive and prone to missing the exact physical surfaces, resulting in visual clipping or "floating" boxes.

By employing Inverse Kinematics, the target (X, Y) coordinates of the end-effector are treated as the inputs. The IK solver uses geometric relationships (the Law of Cosines) to analytically compute the exact crank angles (θ_2, θ_5) required to reach those coordinates. This guarantees mathematically perfect placement of the boxes on the target surfaces.

2.2 IK Solution Implementation

The `solve_IK` function decouples the 6-bar linkage into two separate loops. It calculates the necessary angles by determining the straight-line distance to the target for both the left arm (r_2, r_3) and the right arm (r_5, r_4) .

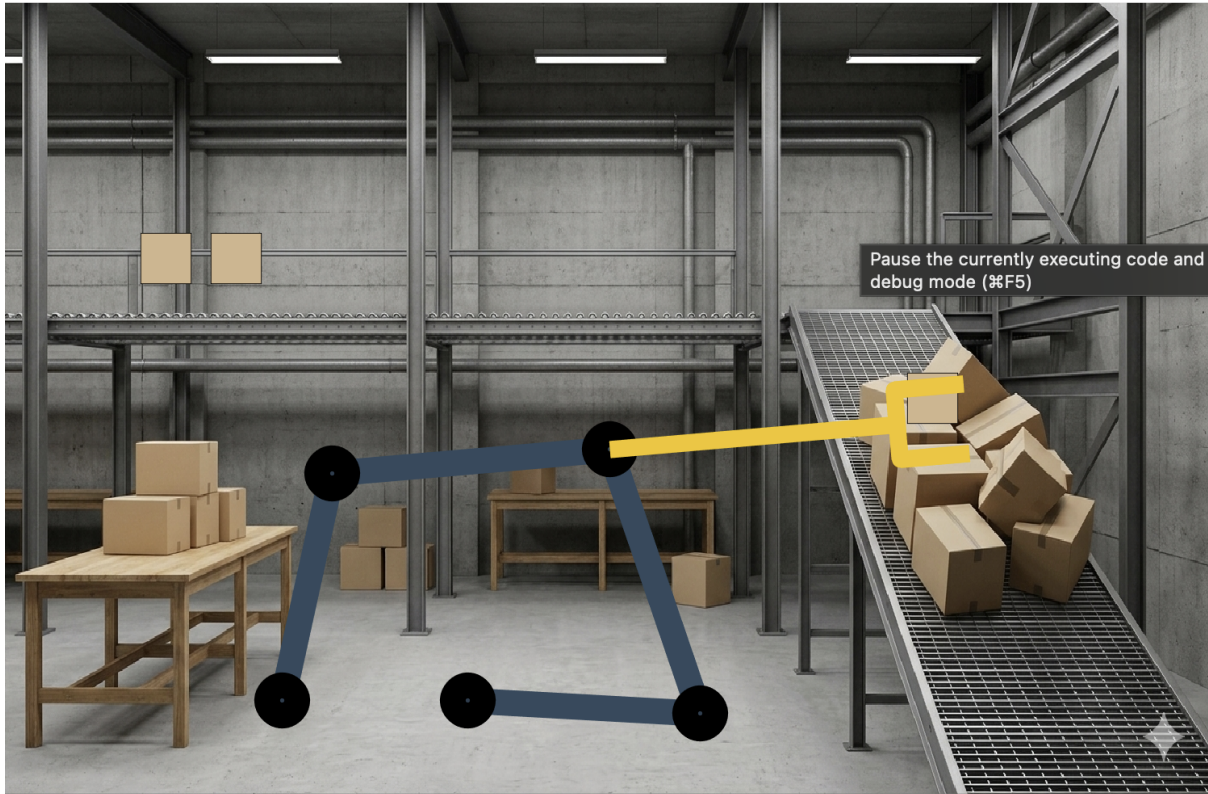


Figure 1: Final layout of the 6-bar linkage within the factory coordinate frame.

```

1 function [t2, t3, t4, t5] = solve_IK(X, Y, r1, r2, r3, r4, r5)
2     % Left Arm
3     D1 = sqrt(X^2 + Y^2);
4     alpha1 = acos(real((r2^2 + D1^2 - r3^2) / (2 * r2 * D1)));
5     t2 = atan2(Y, X) + alpha1;
6
7     % Right Arm
8     X2 = X - r1;
9     D2 = sqrt(X2^2 + Y^2);
10    alpha2 = acos(real((r5^2 + D2^2 - r4^2) / (2 * r5 * D2)));
11    t5 = atan2(Y, X2) - alpha2;
12
13    % Passive Angles
14    J2x = r2 * cos(t2);    J2y = r2 * sin(t2);
15    J4x = r1 + r5*cos(t5); J4y = r5 * sin(t5);
16
17    t3 = atan2(Y - J2y, X - J2x);
18    t4 = atan2(Y - J4y, X - J4x);
19 end

```

Listing 1: Inverse Kinematics Solver (solve_IK.m)

2.3 Forward Kinematics for Visualization

While IK drives the path planning, Forward Kinematics is still utilized inside the `get_coordinates` function. Once `solve_IK` determines the necessary angles, `get_coordinates` computes the spatial coordinates of every single joint to plot the physical lines of the robot in the MATLAB figure.

```

1 function [X, Y] = get_coordinates(t2, t3, t5, r1, r2, r3, r5, r6)
2     O1x = 0;                O1y = 0;

```

```

3      J2x = r2*cos(t2);      J2y = r2*sin(t2);
4      J5x = J2x + r3*cos(t3); J5y = J2y + r3*sin(t3);
5      EEx = J5x + r6*cos(t3); EEy = J5y + r6*sin(t3);
6
7      O2x = r1;      O2y = 0;
8      J4x = O2x + r5*cos(t5); J4y = O2y + r5*sin(t5);
9
10     X = [O1x, J2x, J5x, EEx, J5x, J4x, O2x, O1x];
11     Y = [O1y, J2y, J5y, EEy, J5y, J4y, O2y, O1y];
12 end

```

Listing 2: Coordinate Generator (get_coordinates.m)

3 Design Process and Iterative Refinement

The development of this simulation required resolving several complex kinematic constraints and visual bugs.

3.1 Phase 1: Overshoot and the Staircase Effect

Initially, the simulation experienced "snapping" where boxes would teleport when picked up. The slider (r_6) was extending past the intended target. Furthermore, dropping boxes onto a flat conveyor resulted in a "staircase effect" because a constant plunge depth r_6 interacts differently depending on the internal arm angle θ_3 . This was resolved by computing exact required plunges dynamically: $r_6 = \Delta Y / \sin(\theta_3)$.

3.2 Phase 2: Kinematic Inversion (The Boomerang Effect)

When pushing the arm to the far right to drop the boxes, the top coupler flattened out entirely. Because the prismatic slider runs parallel to the coupler, the linkage reached a singularity where the horizontal thrust caused the tip to "boomerang" backward, dropping multiple boxes in the exact same location.

3.3 Phase 3: Workspace Limits and Mathematical Divergence

When adapting the targets to a wider background, the required reach exceeded the physical link lengths ($D > r_2 + r_3$), resulting in the `acos()` function generating imaginary numbers and crashing the simulation. An attempt to fix this via an iterative precision targeting loop resulted in divergent feedback: the arm flattened out, forcing the slider backward, causing the loop to overcorrect into negative infinity.

3.4 Phase 4: The Final Workspace Scanner

To permanently solve divergence and workspace violations, the base of the robot was relocated back to the origin, taking advantage of the compact dimensions. A brute-force 2D Grid Scanner was implemented before the animation loop. This scanner sweeps thousands of safe, geometrically validated physical coordinates ($D \leq \text{max_reach}$) to find the perfect microscopic elbow position that allows the slider to plunge upward and forward natively into the targets without breaking the linkage.

4 Conclusion

The final MATLAB simulation successfully models a highly precise 6-bar pick-and-place mechanism. By relying on Inverse Kinematics and implementing a geometric workspace scanner,

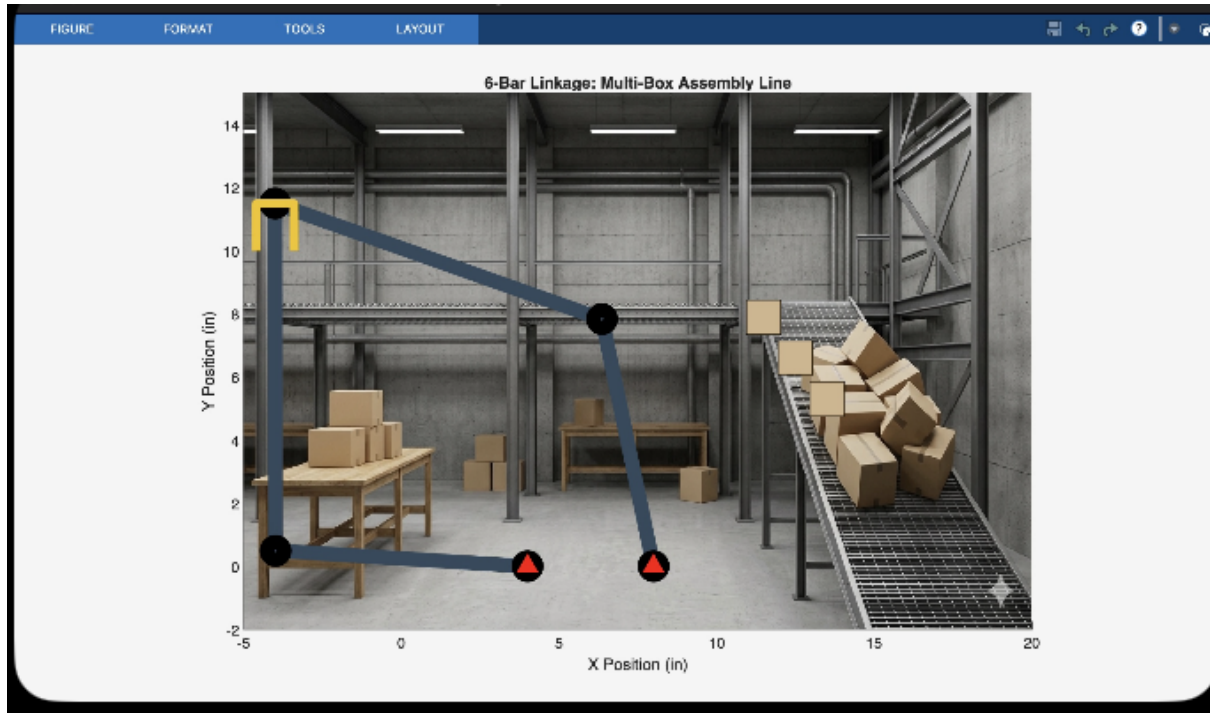


Figure 2: Visual desync caused by early fixed-plunge geometry.

the mechanism avoids singular inversions, completely eliminates divergent targeting loops, and accurately respects the physical limitations of its link lengths.

5 References

1. ENGR 2120 Dynamics and Kinematics, “8 Bar Programming,” *YouTube*, Apr 2, 2023. Available: <https://www.youtube.com/watch?v=2J-r2-KK-G8&list=PLTMo3nrl2KWbeCx8-OGfrMHcFOUSjyk4D&index=9>
2. ENGR 2120 Dynamics and Kinematics, “Tepee Example Part 6 - Newton’s Method,” *YouTube*, Mar 10, 2023. Available: https://www.youtube.com/watch?v=_Ikq9nQTrz4&list=PLTMo3nrl2KWbeCx8-OGfrMHcFOUSjyk4D&index=8
3. A. Haber, “Solve Inverse Kinematics of Robots in MATLAB,” *YouTube*, Apr 15, 2022. Available: <https://www.youtube.com/watch?v=Ry0TiVmAfQA>